

The background of the slide is a soft-focus photograph of numerous white heather flowers. The flowers are small, bell-shaped, and arranged in dense, upright clusters on thin green stems. The lighting is bright and even, creating a clean, minimalist aesthetic. The text is centered over this floral background.

INF.03-05-25.06-SG

Autor: Małgorzata Tokarek

Do wykonania jest witryna:



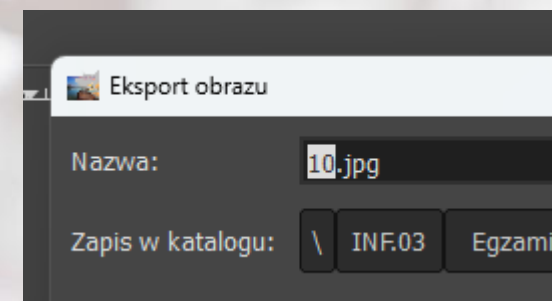
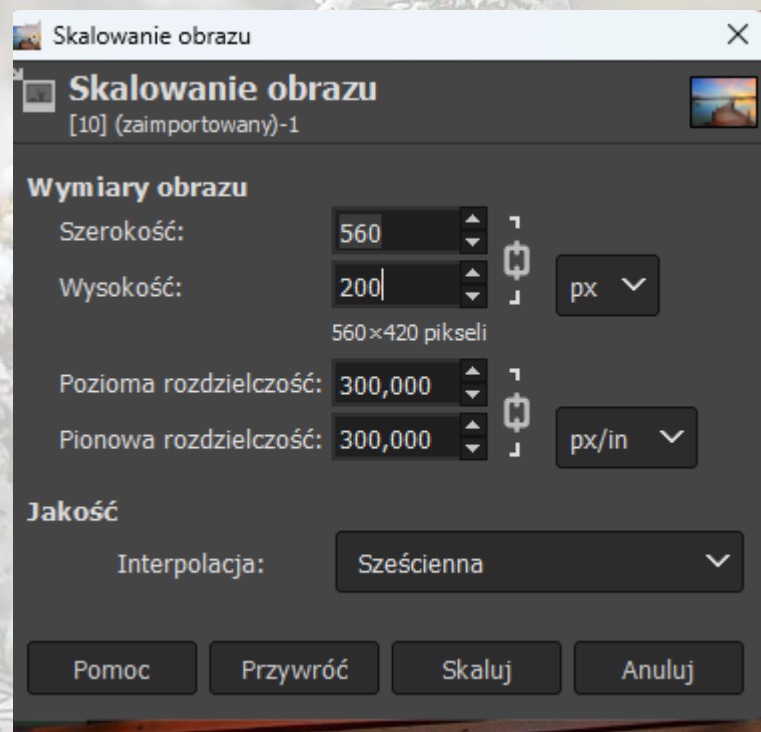
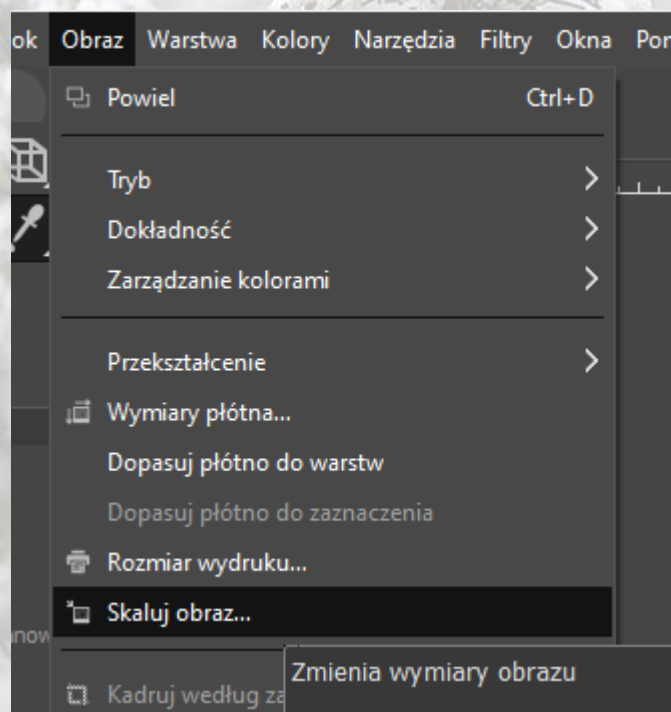
Cechy witryny:

- Składa się z podstron o nazwach *zamowienie.html*, *regulamin.html*, *uslugi.html*
- W pliku *regulamin.html* należy zapisać jedynie: „Regulamin”
- W pliku *uslugi.html* należy zapisać jedynie: „Usługi”

Grafika:

Cechy grafiki 10.jpg:

- Grafikę należy przeskalować z zachowaniem proporcji do wysokości 200 px



Cechy strony *zamowienie.html*:

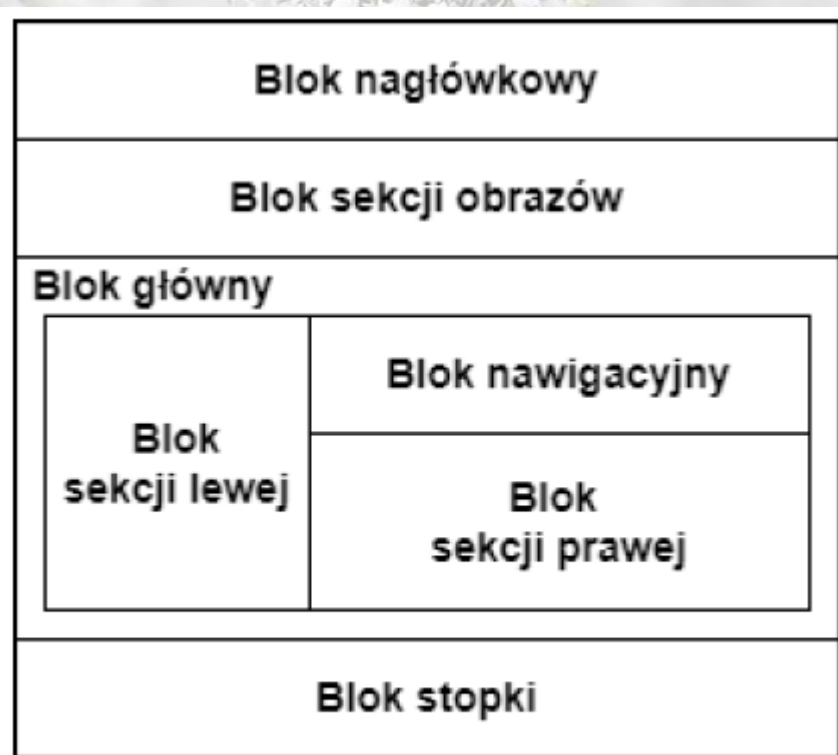
Cechy strony *zamowienie.html*:

- Zapisana w języku HTML5
- Zadeklarowany polski język zawartości witryny
- Jawnie zastosowany właściwy standard kodowania polskich znaków
- Tytuł strony widoczny na karcie przeglądarki: „Fotografia”
- Arkusz stylów w pliku o nazwie *style.css* prawidłowo połączony z kodem strony

```
<!DOCTYPE html>
<html lang="pl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Fotografia</title>
  <link rel="stylesheet"
        href="style.css">
</head>
```

zamowienie.html – podział strony na bloki

Podział strony na bloki zrealizowany za pomocą semantycznych znaczników bloków języka HTML5 tak, aby po uruchomieniu w przeglądarce układ bloków na stronie był zgodny z ilustracją 3



Ilustracja 3. Układ bloków

zamowienie.html – podział strony na bloki

Podział strony na bloki zrealizowany za pomocą semantycznych znaczników bloków języka HTML5 tak, aby po uruchomieniu w przeglądarce układ bloków na stronie był zgodny z ilustracją 3

Pewną wątpliwość wzbudza blok narysowany jako blok boczny i opisany na rysunku jako blok sekcji lewej. Należy jednak trzymać się opisu i koniecznie wykorzystać znacznik *section*, a nie *aside*.



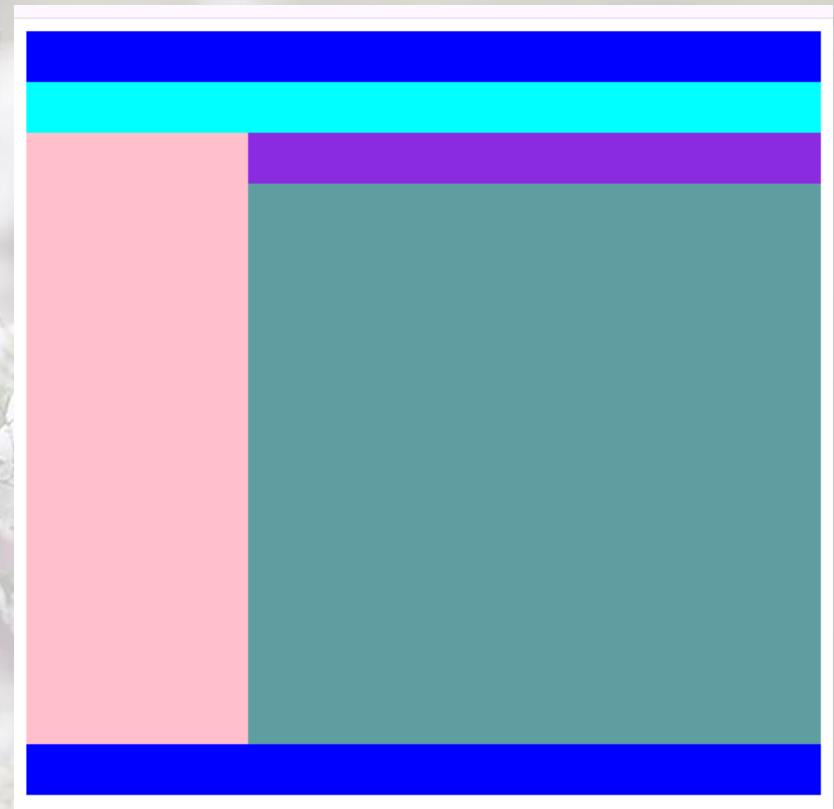
Ilustracja 3. Układ bloków

zamowienie.html – podział strony na bloki

Po napisaniu dla rozwiania wątpliwości w czasie nauki można sobie szybko losowo wystylizować bloki i potwierdzić układ, następnie wykasować testową stylizację :

```
<header></header>
<section id="obrazy"></section>
<main>
  <section id="lewy"></section>
  <section id="naPrawe">
    <nav></nav>
    <section id="prawy"></section>
  </section>
</main>
<footer></footer>
```

Autor: Małgorzata Tokarek



zamowienie.html – dwa pierwsze bloki

Zawartość bloku nagłówkowego: nagłówek pierwszego stopnia o treści „Fotografia artystyczna”

Zawartość bloku sekcji obrazów: 10 obrazów od 1.jpg do 10.jpg. Wszystkie obrazy mają tekst alternatywny „obrazy do sprzedaży” oraz tekst dymka odpowiedzi taki sam jak nazwa pliku, np. 1.jpg

Domyślnym atrybutem HTML używanym do wyświetlania prostego dymka (podpowiedzi) przy najechaniu kursorem na obrazek jest atrybut *title*. Zatem wstawiane obrazy muszą mieć co najmniej trzy atrybuty.

```

```

zamowienie.html – dwa pierwsze bloki

```
<header>
```

```
  <h1>Fotografia artystyczna</h1>
```

```
</header>
```

```
<section id="obrazy">
```

```
  
```

```
  
```

```
  
```

```
  
```

```
  
```

```
  
```

```
  
```

```
  
```

```
  
```

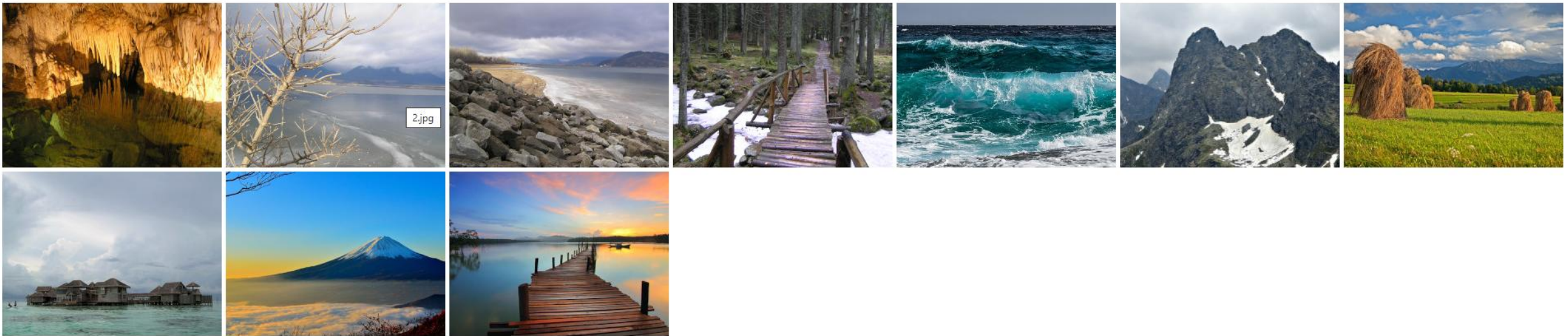
```
  
```

```
</section>
```

zamowienie.html – dwa pierwsze bloki

Aktualny wygląd strony z kursorem na obrazie 2

Fotografia artystyczna



zamowienie.html – sekcja lewa

Nagłówek trzeciego stopnia o treści „Zamówienie”

Etykieta „Wybierz obraz: ”, powiązana z polem edycyjnym znajdującym się obok

Pole edycyjne przeznaczone do wyboru pliku. Akceptuje tylko obrazy w formacie JPEG

Poniżej etykieta „Liczba Kopii: ”, powiązana z polem edycyjnym znajdującym się obok

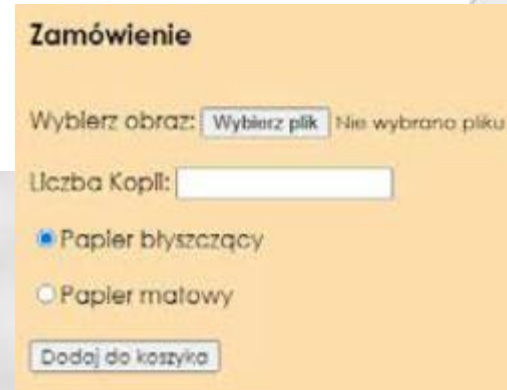
Pole edycyjne przeznaczone do wpisywania jedynie liczb

Poniżej pole opcji „Papier błyszczący”, domyślnie zaznaczone

Poniżej pole opcji „Papier matowy”

W jednym momencie można zaznaczyć tylko jedno pole opcji

Przycisk „Dodaj do koszyka”, którego kliknięcie wywołuje skrypt



The screenshot shows a form titled "Zamówienie" with a light orange background. It contains the following elements: a label "Wybierz obraz:" followed by a "Wybierz plik" button and the text "Nie wybrano pliku"; a label "Liczba Kopii:" followed by a text input field; two radio button options, "Papier błyszczący" (which is selected) and "Papier matowy"; and a "Dodaj do koszyka" button at the bottom.

zamowienie.html – sekcja lewa - etykiety

Etykiety tworzymy znacznikiem *label*. Potrzebuje on atrybutu *for*, aby go powiązać z danym polem edycyjnym w formularzu (kontrolką). Jeśli chcemy aby etykiety miały tą samą szerokość należy pamiętać o *display:inline-block;* w CSS

```
<label for = "imie"           class="szerokosc120">Imię:</label>
<input id  = "imie"          type="text" >
<br><br>
<label for = "nazwisko" class="szerokosc120">Nazwisko:</label>
<input id  = "nazwisko" type="text" >
```

```
.szerokosc120{
  width: 120px;
  display: inline-block;
}
```

Imię:	
Nazwisko:	

zamowienie.html – sekcja lewa - kontrolki

Atrybuty specyficzne tylko dla *input type="file"* :

Atrybut	Opis	Przykład użycia
accept	Określa typy plików, które użytkownik może wybrać. Przeglądarka wyświetli tylko pasujące pliki w oknie dialogowym wyboru.	<code><input type="file" accept=".jpg, .png, .gif"></code>
multiple	Pozwala użytkownikowi wybrać wiele plików jednocześnie. Bez tego atrybutu można wybrać tylko jeden plik.	<code><input type="file" multiple></code>
capture	Tylko dla urządzeń mobilnych. Sugeruje, skąd przeglądarka powinna pozyskać plik (np. bezpośrednio z kamery, a nie z galerii).	<code><input type="file" accept="image/*" capture="user"></code>

W tym zestawie potrzebny będzie atrybut *accept*

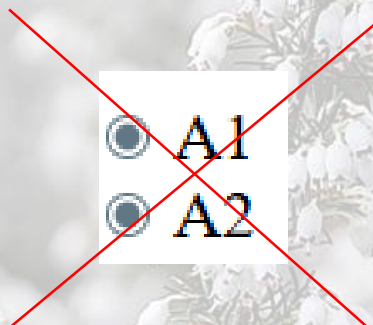
zamowienie.html – sekcja lewa - kontrolki

Przykładowe wartości atrybutu *accept* dla pola *input type=„file”* :

Rodzaje plików	Zapis
Tylko obrazy	<code><input type="file" accept="image/*"></code>
Tylko filmy	<code><input type="file" accept="video/*"></code>
Tylko pliki audio	<code><input type="file" accept="audio/*"></code>
Tylko word i pdf	<code><input type="file" accept=".pdf, .doc, .docx"></code>
Tylko wybrane audio	<code><input type="file" accept=".mp3, .wav, .ogg"></code>
Tylko wybrane obrazy	<code><input type="file" accept=„.jpg, .jpeg"></code>

zamowienie.html – sekcja lewa - kontrolki

Aby nie dało się zaznaczyć dwóch kontrolek radio



należy ustawić im tę samą wartość atrybutu *name*.

Aby zaznaczyć domyślny wybór jednej z kontrolek należy jej nadać atrybut *checked*

zamowienie.html – sekcja lewa

```
<section id="lewy">
  <h3>Zamówienie</h3>
  <label for="plikObrazu">Wybierz obraz:</label>
  <input type="file" id="plikObrazu" accept=".jpg, .jpeg">
  <br>
  <label for="ileKopii">Liczba Kopii:</label>
  <input type="number" id="ileKopii">
  <br>
  <input type="radio" name="papier" id="blysk" checked> Papier błyszczący <br>
  <input type="radio" name="papier" id="mat"> Papier matowy <br>

  <input type="button" value="Dodaj do koszyka" onclick="dodaj()">
</section>
```

Zamówienie

Wybierz obraz: Wybierz plik Nie wybrano pliku

Liczba Kopii:

☒ Papier błyszczący

☐ Papier matowy

zamowienie.html – blok nawigacyjny

- Lista punktowana (nieuporządkowana) z elementami:
 - Odnośnik prowadzący do podstrony *regulamin.html* o treści „Regulamin”
 - Odnośnik prowadzący do podstrony *uslugi.html* o treści „Usługi”
 - Odnośnik prowadzący do strony internetowej *https://pixabay.com* o treści „Zdjęcia”

[Regulamin](#) [Usługi](#) [Zdjęcia](#)

```
<nav>
  <a href="regulamin.html">      Regulamin</a>
  <a href="uslugi.html">        Usługi    </a>
  <a href="https://pixabay.com/"> Zdjęcia  </a>
</nav>
```

zamowienie.html – sekcja prawa

Nagłówek trzeciego stopnia o treści „Koszyk”

Blok, w którym zostanie umieszczony efekt działania skryptu

Wykonywany po stronie klienta, po kliknięciu przycisku

Należy stosować znaczące nazewnictwo zmiennych i funkcji w języku polskim lub angielskim

Pobiera dane z kontrolek

Oblicza cenę na podstawie liczby kopii i rodzaju papieru. Dla papieru błyszczącego cena jednostkowa wynosi 1,5 zł, dla papieru matowego – 2 zł

Ustala nazwę pliku z wartości pobranej z pierwszego pola edycyjnego

Tworzy elementy i dodaje je do bloku z sekcji prawej (ilustracja 4):

- Element DOM dla obrazu z ustaloną nazwą pliku
- Element DOM dla paragrafu z treścią „Liczba kopii: <kopie>”, gdzie pole <> jest pobrane z kontrolki
- Element DOM dla paragrafu z treścią „Cena: <cena>”, gdzie pole <> jest wyliczoną ceną

zamowienie.html – sekcja prawa

Nagłówek trzeciego stopnia o treści „Koszyk”

Blok, w którym zostanie umieszczony efekt działania skryptu

Wymagania dotyczące skryptu:

Wykonywany po stronie klienta, po kliknięciu przycisku

Należy stosować znaczące nazewnictwo zmiennych i funkcji w języku polskim lub angielskim

Pobiera dane z kontrolek

Oblicza cenę na podstawie liczby kopii i rodzaju papieru. Dla papieru błyszczącego cena jednostkowa wynosi 1,5 zł, dla papieru matowego – 2 zł

Ustala nazwę pliku z wartości pobranej z pierwszego pola edycyjnego

Tworzy elementy i dodaje je do bloku z sekcji prawej (ilustracja 4):

- Element DOM dla obrazu z ustaloną nazwą pliku
- Element DOM dla paragrafu z treścią „Liczba kopii: <kopie>”, gdzie pole <> jest pobrane z kontrolki
- Element DOM dla paragrafu z treścią „Cena: <cena>”, gdzie pole <> jest wyliczoną ceną

zamowienie.html – sekcja prawa

Ustalanie ceny za sztukę można zrobić na dwa sposoby:

- 1 sposób - za pomocą *if*

```
let zaznaczoneM = document.getElementById("mat").checked;  
let papierCena = 1.5;  
if (zaznaczoneM)  
    papierCena = 2;
```

- 2 sposób - za pomocą *instrukcji warunkowej*

```
let zaznaczone = document.getElementById("blysk").checked;  
let papierCena = (zaznaczone)? 1.5 : 2 ;
```

zamowienie.html – skrypt – funkcja *dodaj()*

Część 1 kodu

```
//obliczanie ceny za zdjęcia
let ileKopiiElem = document.getElementById("ileKopii")
let ileKopii      = Number(ileKopiiElem.value);
let zaznaczone    = document.getElementById("blysk").checked;
let papierCena    = (zaznaczone)? 1.5 : 2 ;
let cena          = ileKopii * papierCena;
```

zamowienie.html – skrypt – funkcja *dodaj()*

Część 2 kodu

W tej części pobierana jest nazwa pliku z formularza `.files[0].name;`

i na jej podstawie tworzona pełna nazwa pliku

zawierającej się w podfolderze *Obrazy* `= 'Obrazy/' +`

```
//pobranie nazwy zdjęcia  
let obrazElem    = document.getElementById("plikObrazu");  
let obrazNazwa   = 'Obrazy/' + obrazElem.files[0].name;
```

Podobieństwa i różnice JS i PHP w pobieraniu nazw plików

W obu językach nazwa wysyłanego pliku trafia do tablicy mającej w nazwie „pliki”. Gdy wysyła się tylko jedno zdjęcie to tablica plików files w JS ma tylko jeden element, stąd indeks 0. W PHP tablica \$_FILES jest asocjacyjna i dlatego zawsze potrzebuje klucza.

PHP	JS
<code>\$_FILES['fota']['name'];</code>	<code>.files[0].name;</code>

zamowienie.html – skrypt – funkcja *dodaj()*

Część 3 kodu

```
//pobranie miejsca gdzie będzie wstawiany  
//nowy element  
let koszyk = document.getElementById("prawy");
```

zamowienie.html – skrypt – funkcja *dodaj()*

Część 4 kodu

```
//utworzenie i stylizacja pojedynczego diva  
//na info o jednym zdjęciu w koszyku oraz  
//wstawienie go do bloku prawego (koszyka)  
let wKoszyku = document.createElement("div");  
wKoszyku.classList.add("wKoszyku");  
koszyk.appendChild(wKoszyku);
```

zamowienie.html – skrypt – funkcja *dodaj()*

Część 5 kodu

```
//utworzenie zdjęcia, nadanie mu atrybutów  
//oraz wstawienie do pojemnika wKoszyku  
let obraz = document.createElement("img");  
obraz.src = obrazNazwa;  
wKoszyku.appendChild(obraz);
```

zamowienie.html – skrypt – funkcja *dodaj()*

Część 6 kodu

```
//utworzenie pojemnika na dwa akapity
//utworzenie dwóch akapitów
let naAkapity = document.createElement("div");
let akapit1   = document.createElement("p");
let akapit2   = document.createElement("p");
//wpisanie treści do akapitów
akapit1.innerHTML = "Liczba kopii: " + ileKopii;
akapit2.innerHTML = "Cena: " + cena;
```

zamowienie.html – skrypt – funkcja *dodaj()*

Część 7 kodu

```
//umieszczenie akapitów w nowym pojemniku  
//umieszczenie pojemnika z akapitami w wKoszyku  
naAkapity.appendChild(akapit1);  
naAkapity.appendChild(akapit2);  
wKoszyku.appendChild(naAkapity);
```

zamowienie.html

skrypt i funkcja *dodaj()*

Autor: Małgorzata Tokarek

```
function dodaj()  
    //obliczanie ceny za zdjęcie  
    let ileKopiiElem = document.getElementById("ileKopii")  
    let ileKopii     = Number(ileKopiiElem.value);  
    let zaznaczone   = document.getElementById("blysk").checked;  
    let papierCena   = (zaznaczone)? 1.5 : 2 ;  
    let cena         = ileKopii * papierCena;  
  
    //pobranie nazwy zdjęcia  
    let obrazElem   = document.getElementById("plikObrazu");  
    let obrazNazwa   = 'Obrazy/' + obrazElem.files[0].name;  
  
    //pobranie miejsca gdzie będzie wstawiany  
    //nowy element  
    let koszyk = document.getElementById("prawy");  
  
    //utworzenie i stylizacja pojedynczego diva  
    //na info o jednym zdjęciu w koszyku oraz  
    //wstawienie go do bloku prawego (koszyka)  
    let wKoszyku = document.createElement("div");  
    wKoszyku.classList.add("wKoszyku");  
    koszyk.appendChild(wKoszyku);  
  
    //utworzenie zdjęcia, nadanie mu atrybutów  
    //oraz wstawienie do pojemnika wKoszyku  
    let obraz = document.createElement("img");  
    obraz.src = obrazNazwa;  
    wKoszyku.appendChild(obraz);  
  
    //utworzenie pojemnika na dwa akapity  
    //utworzenie dwóch akapitów  
    let naAkapity = document.createElement("div");  
    let akapit1   = document.createElement("p");  
    let akapit2   = document.createElement("p");  
    //wpisanie treści do akapitów  
    akapit1.innerHTML = "Liczba kopii: " + ileKopii;  
    akapit2.innerHTML = "Cena: " + cena;  
  
    //umieszczenie akapitów w nowym pojemniku  
    //umieszczenie pojemnika z akapitami w wKoszyku  
    naAkapity.appendChild(akapit1);  
    naAkapity.appendChild(akapit2);  
    wKoszyku.appendChild(naAkapity);
```

zamowienie.html - stopka

Zawartość stopki: paragraf o treści „Autor strony: ”, dalej wstawiony numer zdającego. Numer zdającego jest zapisany za pomocą znacznika semantycznego oznaczającego tekst uwypuklony, formatowany domyślnie jako pochylony

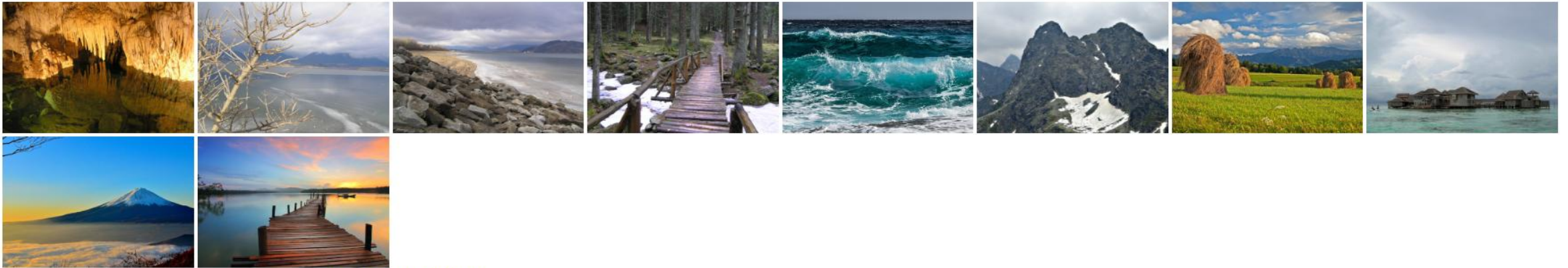
```
<footer>
  <p>
    Autor strony:
    <em>007</em>
  </p>
</footer>
```

Autor: Małgorzata Tokarek

zamowienie.html - aktualny wygląd

Wygląd bez CSS za wyjątkiem dołożonych reguł *display: flex;*

Fotografia artystyczna



[Regulamin Usługi Zdjęcia](#)

Zamówienie

Wybierz obraz: 9.jpg

Liczba Kopii:

- ☒ Papier błyszczący
☐ Papier matowy

Koszyk



Liczba kopii: 2

Cena: 4

Liczba kopii: 4

Cena: 6

CSS

Domyślnie, dla wszystkich selektorów: krój czcionki Century Gothic, w przypadku braku Arial
Dla ciała strony: kolor tła BlanchedAlmond

Uwaga: Na egzaminie próbnym było tu dużo błędów. Uczniowie szukali, którą czcionkę mają i tą wpisywali, a tu chodzi o zapis obu i to we właściwej kolejności

```
*{  
    font-family: Century Gothic, Arial;  
}  
body{  
    background-color: BlanchedAlmond;  
}
```

Autor: Małgorzata Tokarek

Dla bloku nagłówkowego i stopki: kolor tła Sienna, biały kolor czcionki, marginesy wewnętrzne 5 px, wyrównanie tekstu do środka, rozmiar czcionki 130%

Dla sekcji lewej: kolor tła NavajoWhite, marginesy zewnętrzne 10 px, wewnętrzne 20 px, szerokość 35%, wysokość 400 px, cień o przesunięciu 5 px w obu osiach, rozmyciu 10 px i kolorze DimGray

Dla bloku nawigacyjnego: szerokość 50%, wysokość 50 px

Dla sekcji prawej: szerokość 50%, wysokość 420 px, paski przewijania pojawiające się tylko w przypadku przepełnienia bloku

```
header, footer{
    background-color: ■ sienna;
    color: □ white;
    padding: 5px;
    font-size: 130%;
    text-align: center;
}

#lewy{
    background-color: ■ NavajoWhite;
    margin: 10px;
    padding: 20px;
    width: 35%;
    height: 400px;
    box-shadow: 5px 5px 10px ■ DimGray;
}
```

CSS

```
nav{
    width: 50%;
    height: 50px;
}

#prawy{
    width: 50%;
    height: 420px;
    overflow: auto;
}
```

CSS

Dla pola edycyjnego i przycisku: jedynie zewnętrzny margines górny 20 px

Dla selektora elementu listy: sposób wyświetlania liniowo-blokowy

Dla selektora obrazu: szerokość 10%, opływanie po prawej stronie (obraz po lewej stronie)

Dla selektora odnośnika: kolor czcionki Sienna, jedynie zewnętrzny margines prawy 30 px

W momencie, gdy kursor znajdzie się na odnośniku jego kolor czcionki zamienia się na #4C1900

```
input[type="file"], input[type="number"], input[type="button"]{  
    margin-top: 20px;  
}  
  
ul li {  
    display: inline-block;  
}  
  
img{  
    width: 10%;  
    float:left;  
}  
  
.czysc{  
    clear:both;  
}  
  
a{  
    color: ■ sienna;  
    margin-right: 30%;  
}  
  
a:hover{  
    color: ■ #4C1900;  
}
```

Przy tej stylizacji w koszyku
robi się układ:

Koszyk



Liczba kopii: 3
Cena: 4.5



Liczba kopii: 35
Cena: 52.5

CSS

Błąd wizualny z poprzedniego slajdu można obejść stosując, ale nie jest punktowane rozwiązanie.

```
.wKoszyku{  
  display: flex;  
}
```

Koszyk



Liczba kopii: 1
Cena: 1.5

Liczba kopii: 1
Cena: 1.5

CSS – *float*: i *clear:both*;

Stosując *float:left*; (lub *float:right*;) należy zawsze dołożyć na początku nowego pojemnika *clear:both*; . Nawet jeśli dobrze działa wyrównywanie w górnej części to i tak trzeba dołożyć pojemnik z tą stylizacją:

```




<div class="czysc"></div>
</section>
```

```
.czysc{
    clear:both;
}
```

CSS – *float*: i *clear:both*;

W koszyku nasze obrazy są *float:left*; bo wymusiły to na nas dyspozycje CSS zestawu. Należy zatem *clear:both*; . W tym przypadku oznacza to, że każdy nowo dodany komponent (div wKoszyku) musi być *clear:both*;

```
.wKoszyku{  
    clear:both;  
}
```

Koszyk



Liczba kopii: 2
Cena: 3



Liczba kopii: 2
Cena: 3

CSS – *float*: i *clear:both*;

Aby z pewnością nie stracić punktów za jakiś brakujący *clear:both*;

warto po pojemniku koszyka (section id=„prawy”) dodać pojemnik czyszczący.

W ten sposób z prawej strony mamy trzy pojemniki:

```
<section id="naPrawe">
```

```
  <nav> ...
```

```
</nav>
```

```
  <section id="prawy">
```

```
    <h3>Koszyk</h3>
```

```
  </section>
```

```
  <div class="czysc"></div>
```

```
</section>
```

1. Blok nawigacyjny

2. Blok sekcji prawej

3. Blok czyszczący ustawienia float

CSS – nieścisłości i różnice:

CSS – zrobione według wskazań z arkusza

[Regulamin Usługi Zdjęcia](#)

Zamówienie

Wybierz obraz: 7.jpg

Liczba Kopii:

☒ Papier błyszczący
☐ Papier matowy

Koszyk

	Liczba kopii: 10 Cena: 20
	Liczba kopii: 7 Cena: 14
	Liczba kopii: 20 Cena: 30

Przykład z arkusza – zdjęcia z arkusza

[Regulamin](#) [Usługi](#)

Zamówienie

Wybierz obraz: 7.jpg

Liczba Kopii:

☒ Papier błyszczący
☐ Papier matowy

Koszyk

	Liczba kopii: 10 Cena: 20
	Liczba kopii: 7 Cena: 14
	Liczba kopii: 20 Cena: 30

CSS – nieścisłości i różnice:

Dla pola edycyjnego i przycisku: jedynie zewnętrzny margines górny 20 px

Pole edycyjne przeznaczon
Poniżej etykieta „Liczba Kop
Pole edycyjne przeznaczon
Poniżej pole opcji „Papier bl
Poniżej pole opcji „Papier m

Stylizacja przycisków radio ??? W opisie HTML i CSS pola radio nazwane są przyciskami opcji, a w CSS piszą jedynie o stylizacji pola edycyjnego i przycisku:

```
input[type="file"], input[type="number"], input[type="button"]{  
    margin-top: 20px;  
}
```

Strona wygląda jednak jakby stylizacja dotyczyła wszystkich inputów i przy tej drugiej stylizacji mamy zgodność wyglądu:

```
input{  
    margin-top: 20px;  
}
```

Autor: Małgorzata Tokarek

Wybierz obraz: Nie w

Liczba Kopii:

☒ Papier błyszczący

☐ Papier matowy

CSS – nieścisłości i różnice – radio – cd.

Radio jest polem opcji, a nie edycyjnym dlatego dla bezpieczeństwa zrobiłabym:

```
input[type="file"], input[type="number"], input[type="button"]{  
    margin-top: 20px;  
}
```

```
/*
```

Aby zachować zgodność wyglądu
dołożyłabym stylizację pola opcji:

```
input[type="radio"]{  
    margin-top: 20px;  
}
```

```
*/
```

CSS – nieścisłości i różnice

Druga różnica dotyczy położenia bloków z prawej strony.

35%(sekcja lewa) + 50%(nav, sekcja prawa i czyszcząca) = 85% ekranu

Wygląda jakby brakujące 15% było marginesem dla naszego pojemnika *naPrawe*:

Po dołożeniu takiego marginesu układ się poprawia:

Regulamin Usługi Zdjęcia

Zamówienie

Wybierz obraz: 7.jpg

Liczba Kopii:

☒ Papier błyszczący
☐ Papier matowy

Koszyk

	Liczba kopii: 7 Cena: 14
	Liczba kopii: 10 Cena: 20
	Liczba kopii: 20 Cena: 30

Autor strony: 007
Autor: Małgorzata Tokarek

```
#naPrawe{  
    margin-left: 15%;  
}
```

CSS – nieścisłości i różnice

Pozostaje jeszcze kwestia odległości w pionie między kodem napisanym zgodnie z instrukcją, a wyglądem:

Koszyk



Liczba kopii: 7
Cena: 14



Liczba kopii: 10
Cena: 20



Liczba kopii: 20
Cena: 30

Koszyk



Liczba kopii: 10
Cena: 20



Liczba kopii: 7
Cena: 14



Liczba kopii: 20
Cena: 30

CSS – nieścisłości i różnice

Efekt zbliżony do widocznego na arkuszu da dodatkowa stylizacja elementów klasy *wKoszyku*:

```
.wKoszyku{  
    clear:both;  
    line-height: 200%;  
    margin-top: 20px;  
    margin-bottom: 20px;  
}
```

Z dodatkowymi regułami CSS	Widok w arkuszu
Koszyk Liczba kopii: 10 Cena: 20 Liczba kopii: 7 Cena: 14 Liczba kopii: 20 Cena: 30	Koszyk Liczba kopii: 10 Cena: 20 Liczba kopii: 7 Cena: 14 Liczba kopii: 20 Cena: 30

CSS – z dołożonymi poprawkami

Fotografia artystyczna



[Regulamin Usługi Zdjęcia](#)

Zamówienie

Wybierz obraz: 7.jpg

Liczba Kopii:

☒ Papier błyszczący

☐ Papier matowy

Koszyk



Liczba kopii: 10
Cena: 20



Liczba kopii: 7
Cena: 14



Liczba kopii: 20
Cena: 30

Autor strony: 007

CSS - punktacja

W wytycznych do oceniania nie ma nigdzie punktów do oceniania dołożonych rzeczy, zatem teoretycznie całość punktów powinna być za wystylizowanie zgodnie z wytycznymi bez dodatkowych czynności.

Uproszczenie *dodaj()*

```
//utworzenie dwóch akapitów
let akapit1 = document.createElement("p");
let akapit2 = document.createElement("p");
//wpisanie treści do akapitów
akapit1.innerHTML = "Liczba kopii: " + ileKopii;
akapit2.innerHTML = "Cena: " + cena;

//umieszczenie akapitów w wKoszyku
wKoszyku.appendChild(akapit1);
wKoszyku.appendChild(akapit2);
```

Dodaliśmy kilka divów czyszczącym przez float, możemy też zabrać jeden w funkcji *dodaj()*. Jest to pojemnik *naAkapity*. Można go nie tworzyć bo przez float akapity same dobrze się ułożą.

dodaj() po skróceniu

Autor: Małgorzata Tokarek

```
function dodaj(){  
    //obliczanie ceny za zdjęcie  
    let ileKopiiElem = document.getElementById("ileKopii")  
    let ileKopii     = Number(ileKopiiElem.value);  
    let zaznaczone   = document.getElementById("blysk").checked;  
    let papierCena   = (zaznaczone)? 1.5 : 2 ;  
    let cena         = ileKopii * papierCena;  
  
    //pobranie nazwy zdjęcia  
    let obrazElem   = document.getElementById("plikObrazu");  
    let obrazNazwa   = 'Obrazy/' + obrazElem.files[0].name;  
  
    //pobranie miejsca gdzie będzie wstawiany  
    //nowy element  
    let koszyk = document.getElementById("prawy");  
  
    //utworzenie i stylizacja pojedynczego diva  
    //na info o jednym zdjęciu w koszyku oraz  
    //wstawienie go do bloku prawego (koszyka)  
    let wKoszyku = document.createElement("div");  
    wKoszyku.classList.add("wKoszyku");  
    koszyk.appendChild(wKoszyku);  
  
    //utworzenie zdjęcia, nadanie mu atrybutów  
    //oraz wstawienie do pojemnika wKoszyku  
    let obraz = document.createElement("img");  
    obraz.src = obrazNazwa;  
    wKoszyku.appendChild(obraz);  
  
    //utworzenie dwóch akapitów  
    let akapit1 = document.createElement("p");  
    let akapit2 = document.createElement("p");  
    //wpisanie treści do akapitów  
    akapit1.innerHTML = "Liczba kopii: " + ileKopii;  
    akapit2.innerHTML = "Cena: " + cena;  
  
    //umieszczenie akapitów w wKoszyku  
    wKoszyku.appendChild(akapit1);  
    wKoszyku.appendChild(akapit2);  
}
```